॥श्री गणेशाय नमः॥

Transactions

Transaction:
A transaction is a set of logically related operations to perform a unit of work.
E.g. If a person A needs to send ₹500 to another person B. Operations including Bal_A -= 500, Bal_B += 500 will take place. All of those together are a transaction.

Data item:
It is a DB element that one/more transaction(s) need to access. E.g. an entire DB, a file in a DB, a block of a file or a record in a block.

Main operations in transaction control

Read(X)

This will fetch a data item X from the disk into the main memory to use the value of X in some transaction logic.
Steps:
1. Find the block from the DB file which consists the data item X.
2. Transfer the entire block from the hard disk to the main memory.
3. Find the address of X in the main memory block and copy the value of X in a programmed variable.
(These steps are atomic).

Write(X)

This will update a data item X in the disk
Steps:
1. Find the block from the DB file which consists the data item X.
2. Transfer the entire block from the hard disk to the main memory.
3. Find the address of X in the main memory block and update X in the main memory.
4. Replace the block in the disk with the updated block.
(These steps are atomic).

Consider a table Account(cid, bal). We need to simulate transfer of bal amount 500 from an account with cid=101 to an account with cid=102.

In SQL:

```
BEGIN TRANSACTION T1
    UPDATE Account
    SET bal = bal - 500
    WHERE cid = 101;

    UPDATE Account
    SET bal = bal + 500
    WHERE cid = 102;
COMMIT TRANSACTION T1;
```

Low level:

```
BEGIN TRANSACTION T1
    READ (bal of cid = 101)
    bal = bal - 500
    WRITE (bal of cid = 101)

    READ (bal of cid = 102)
    bal = bal + 500
    WRITE (bal of cid = 102)
COMMIT TRANSACTION T1
```

Set bal of both accounts to 5000.

```
BEGIN TRANSACTION T1
    UPDATE Account
    SET bal = 5000 WHERE cid = 101;
    UPDATE Account
    SET bal = 5000 WHERE cid = 102;
COMMIT TRANSACTION T1
```

```
BEGIN TRANSACTION T1
    WRITE (bal: 5000 WHERE cid = 101);
    WRITE (bal: 5000 WHERE cid = 102);
COMMIT TRANSACTION T1
```

When dealing with such transactions, we care only about

```
WRITE(bal);
READ(bal);
```

We'll represent transactions as something like

```
T1

READ(A)
WRITE(A)
READ(B)
WRITE(B)
WRITE(C)    // blind write
COMMIT
```

ACID properties

Atomicity, durability, isolation, and consistency

- Atomicity: maintained by recovery mgmt component of the DBMS.
- Durability: maintained by concurrency control.
- Isolation, consistency: maintained by the user.

These are required to preserve integrity (correctness of transactions).

Atomicity

Execute all operations of a transaction including commit or execute none of them.

E.g. There's a transaction to transfer 500 from acc A to acc B.

```
Trans (T1)
R(A)
A = A-500
W(A)
R(B)
B = B+500
W(B)
COMMIT
```

Say a failure occurs at this point.
Then 500 is deducted from A, but not added to B.
This failure leads to incorrect data.
To preserve integrity, atomicity is used.
A rollback takes place.

The recovery manager is responsible for the successful rollback of a failed transaction.

Recovery management component: It rolls back a transaction if it fails at any point before committing.

Roll back (abort): Undoing modifications to a DB file done by a failed transaction.

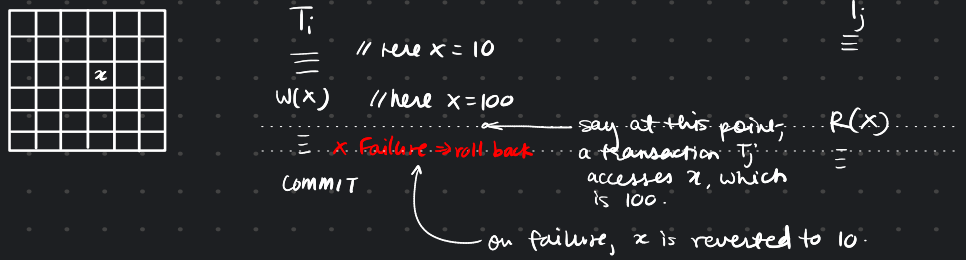
Transaction log

It is a file maintained by the recovery manager in the disk to record all actions done by transactions.

The recovery management component checks the log to perform roll backs.

1.	T_1 R(X)
2	T_2 W(X)

Say there's a value X in the disk. There's a transaction T_i that has performed a WRITE on X. Some failure occurs before COMMIT.



A block updated by an uncommitted transaction is called "dirty". A dirty flag should be maintained and dirty reads should be prevented.

There's a chance of roll back and that some other transaction might read the uncommitted (dirty) data.

Once the transaction commits, a roll back isn't possible. So the block becomes NOT dirty (dirty flag = 0).

Redo operations:

All modifications done on COMMIT.

E.g. removing the dirty flag, flushing data;

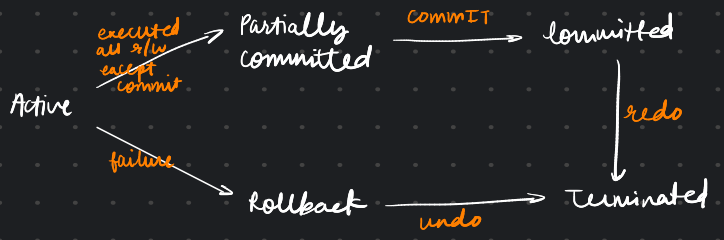
- flushing data: any written data stays in the RAM for a while.

Undo operations:

All operations on a DB file done on roll backs.

E.g. A transaction changes a variable A from 10 to 100, but fails at some further point. On rolling back, A is changed back to 10. The DB reflects its state before the start of the transaction.

States of a transaction



Durability

A transaction should recover under any case of failure.

A transaction may fail due to:

power failures, software crashes(OS overload, DBMS overload), the concurrency control protocol may kill transactions, hardware crashes;

a serious problem,
since the transaction log will be lost
soln: use multiple disks (RAID)

Consistency

It is the user's responsibility to check whether the operations performed by a transaction are logically consistent.

E.g. Say we need to transfer 500 from acc A(current bal = 2000) to acc B(current bal = 1000). The user should make sure that bal(A) + bal (B) stays the same after the transaction occurs.

Isolation

The result of concurrent execution of 2 or more transactions must be equal to result of any serial execution of those transactions.

T_1: $R_1(w)$ $W_1(x)$ $R_1(y)$ $W_1(y)$
T_2: $R_2(a)$ $W_2(b)$ $R_2(b)$

$T_1 \mid T_2$ is same as $T_2 \mid T_1$.

Schedule

It is a time-ordered execution sequence of 2/more transactions. It is a combination of interleaving transactions.

	T1	T2	T3
1	R(A)		
2		R(A)	
3			R(B)
4		W(A)	
5			W(B)
6	W(A)		

A schedule can have any # of transactions.

can be written as

$T_1 : R(A) \mid T_2 : R(A) \mid T_3 : R(B) \mid T_2 : W(A) \mid \ldots$

OR

$R_1(A),\ R_2(A),\ R_3(B),\ W_2(A),\ \ldots$

Serial schedule

The transactions are executed one after the other without interleaving.

T_1 : Transfer 500 from A to B.
$[R_1(A), W_1(A), R_1(B), W_1(B)\ C_1]$

T_2 : Display A+B
$[R_2(A)\ R_2(B)\ C_2]$

Here, there's 2 ways this could happen.

T_1	T_2
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
$w_1(B)$	
commit c_1	
	$r_2(A)$
	$r_2(B)$
	commit c_2

or

T_1	T_2
	$r_2(A)$
	$r_2(B)$
	commit c_2
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
$w_1(B)$	
commit c_1	

Say there are 10 transactions.
of serial schedules possible = 10!

Adv:
Data items in a serial schedule are accessed by one transaction at a time, so there are no inconsistencies.
Disadv:
No concurrency, less throughput. No sharing of resources.

Consider the schedules (assume that dirty reads are allowed):

S_1

T_1	T_2
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
$w_1(B)$	
c_1	
	$r_2(A)$
	$r_2(B)$
	c_2

S_2

T_1	T_2
	$r_2(A)$
	$r_2(B)$
	c_2
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
$w_1(B)$	
c_1	

T_1	T_2
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
	$r_2(B)$
	c_2
$r_1(B)$	
$w_1(B)$	
c_1	

T_1	T_2
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
$r_1(B)$	
$w_1(B)$	
c_1	
	$r_2(B)$
	c_2

(concurrent schedules)

Which of S1, S2, S3, and S4 are equivalent?

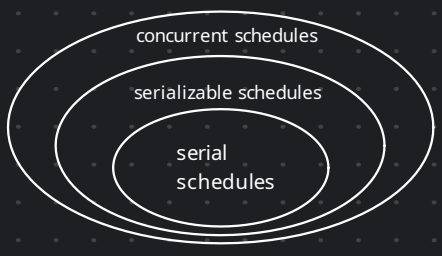
S_1, S_2, and S_4 are equivalent.

→ S_4 is serializable.
S_4 is equivalent to S_1
$\therefore S_1$ is equivalent to any serial schedule of S_1.
S_4 is called serializable.

Serializable schedule:
A schedule is serializable if it is equivaluent to any serial schedule.

Isolation

The result of concurrent schedules should be equal to the result of any serial schedule.
These schedules are called serializable.



Concurrency control protocols

These only allow serializable schedules.

Q. $T_1 : R_1(A)\ W_1(A)\ R_1(B)\ W_1(B)$
$T_2 : R_2(A)\ R_2(B)$
How many concurrent schedules are possible?

→ $\frac{6!}{4!\,2!}$ (6! ← total operation orderings; 4! ← no rearrangements in the first; 2! ← no rearrangements in the second.)

OR ——————— ${}^6C_4 \cdot {}^2C_2$ (↳ for T_1, for T_1)

Q. T_1, T_2 are transactions with m and n operations each. How many concurrent schedules are possible?

→ $\frac{(m+n)!}{m!\,n!}$ OR ${}^{m+n}C_m\ {}^nC_n$

Q. T_1, T_2, T_3 with m, n, p operations each. # of possible concurrent schedules?

→ $\frac{(m+n+p)!}{m!\,n!\,p!}$